

Amortized Deduction for Binary Linear Systems: Learning Branching Guidance from Exact Conditional Marginals

neuroMCTS project^a

^a*Korea University, Seoul, Republic of Korea*

Abstract

A binary linear system (BLS) asks for $\mathbf{x} \in \{0, 1\}^n$ satisfying $A\mathbf{x} = \mathbf{b}$ with $A \in \{0, 1\}^{m \times n}$, a formulation that arises in laser-based non-destructive testing, where a binary occupancy image is reconstructed from integer projections. On market-split-style instances the linear relaxation is deliberately uninformative, so backtracking search spends most of its effort on branching decisions that a cheap heuristic cannot make well. We present a learned branching heuristic for this setting and an exact procedure for training it.

The method rests on two observations. First, conditioning a BLS on a partial assignment is *identical* to substituting the assignment out and obtaining a smaller BLS; a single network therefore predicts at the root and at every interior node without architectural change, and training states are ordinary instances of reduced size. Second, on instances small enough to enumerate, the exact conditional marginal $p_j = \Pr[x_j = 1 \mid A, \mathbf{b}]$ is computable, and it partitions the free variables of a node into those every surviving solution agrees on — where branching the other way empties the subtree and forces a backtrack — and those solutions disagree on, where either branch still reaches a solution. Only the first kind carries cost. This partition supplies both an exact training target and a criterion for which predictions matter: at the root only 6.3% of variables are of the first kind, but by depth 8 the share is 87.9%, which is where prediction accuracy begins converting into search cost.

We train a 21.8K-parameter bipartite graph network on these targets and use it for branching only, never for fixing variables, since it produces a probability rather than a proof. Against LP-probing — the sound procedure that detects the same variables, at one linear program each — the network needs

a single relaxation and one forward pass, so their cost ratio grows linearly in instance size: $1.7\times$ at $n = 25$, $7.2\times$ at $n = 60$, $17.8\times$ at $n = 150$, measured with a warm-started backend on both sides. Under a matched wall-clock budget on a warm-started search loop, replicated over three seeds, the resulting search is $2.6\text{--}3.0\times$ faster in median than LP-guided branching at 21×60 (faster on 23/30 instances in every seed, sign test $p = 0.005$) and expands $9.5\times$ fewer nodes, with both strategies solving every instance; within the full pipeline, where lattice reduction runs first, it closes the residual $3.0\times$ faster than LP-guided branching, while random branching closes none of it. Controlling the number of solutions across sizes, we find no measurable transfer penalty: a network trained only at 10×25 is statistically indistinguishable from one trained at the evaluation size.

We also report what the method does not achieve. CP-SAT solves the same instances $13\text{--}40\times$ faster at every size tested, and our system does not compete with it; decomposing that gap shows our branching produces $2.2\times$ fewer nodes but processes them $32\times$ more slowly, which places the deficit in per-node cost rather than in branching quality, though we do not show that it is recoverable. And the cost advantage over probing, while asymptotically real, is only $1.3\text{--}1.7\times$ at the depths where prediction matters most on our instances, where probing additionally offers proofs rather than predictions. The contribution we claim is therefore methodological: exact conditional posteriors are an obtainable and well-posed supervision signal for branching, in-tree states are a better training distribution than root states — a root-trained network is worse than relaxation rounding when deployed in the tree, an in-tree-trained one is better by 3.4 points — and the resulting heuristic transfers across instance size. Code, generators, exact-marginal labels and per-instance results are released.

Keywords: Binary linear systems, Learning to branch, Amortized optimization, Exact conditional marginals, Market split, Graph neural networks

1. Introduction

1.1. Problem

Let $A \in \{0, 1\}^{m \times n}$ and $\mathbf{b} \in \mathbb{Z}^m$. A *binary linear system* (BLS) is the feasibility problem

$$\text{find } \mathbf{x} \in \{0, 1\}^n \text{ such that } A\mathbf{x} = \mathbf{b}, \quad (1)$$

5 whose solution set we write $\mathcal{S} = \{\mathbf{x} \in \{0, 1\}^n : A\mathbf{x} = \mathbf{b}\}$. Each row is a
6 cardinality constraint $\sum_j a_{ij}x_j = b_i$ stating how many cells of a designated
7 subset are occupied.

8 The motivating application is laser-based non-destructive testing. A spec-
9 imen is scanned along m directions; each scan returns the integer count of
10 absorbing cells along its path, and the task is to reconstruct the binary occu-
11 pancy image. Two properties of that setting shape the problem formulation.
12 Solutions need not be unique — several images can be consistent with the
13 same measurements, and recovering *any* member of $\mathcal{S}$ solves the physical
14 problem. And measurement noise can perturb $\mathbf{b}$ so that $\mathcal{S} = \emptyset$, making a
15 reliable infeasibility verdict as valuable as a solution.

16 Deciding $\mathcal{S} \neq \emptyset$ is NP-complete in general. The instances studied here
17 are drawn from a deliberately hard regime modelled on market-split problems
18 [6, 2], in which the linear relaxation

$$\tilde{\mathcal{S}} = \{\mathbf{x} \in [0, 1]^n : A\mathbf{x} = \mathbf{b}\} \quad (2)$$

19 is a large polytope with predominantly fractional vertices. Relaxation-based
20 bounding is therefore weak, and a complete solver must decide where to
21 branch with little guidance from the relaxation — exactly the decision this
22 paper learns.

23 1.2. Where a learned component attaches, and what to train it on

24 A complete solver for (1) alternates deduction (constraint propagation,
25 relaxation-based pruning) with a *branching decision*: which free variable to
26 split on, and which value to try first. Deduction is sound and must remain
27 so; branching is a heuristic choice, where an error costs search time but never
28 correctness. This asymmetry makes branching the natural place to attach
29 a learned component, and it is where learning has been most successful in
30 mixed-integer programming [9, 7].

31 The difficulty is supervision. Two standard answers are available and
32 both are unsatisfactory here. Training against a planted solution $\mathbf{x}^*$ treats
33 one member of $\mathcal{S}$ as ground truth, which is ill-posed when $|\mathcal{S}| > 1$: nothing
34 in $(A, \mathbf{b})$ distinguishes the planted solution from any other, so the label is a
35 sample from the posterior rather than the posterior. Imitating an expensive
36 expert, as in strong-branching imitation, requires an expert whose decisions
37 are both informative and affordable to collect; the natural candidate here,
38 LP-probing, costs one linear program per variable per node.

We take a third route, available because these instances have planted solutions and because feasibility can be enumerated at moderate size: compute the *exact conditional posterior* by exhaustive enumeration on small instances, and train against it. Section 3 shows that this is both tractable and, in a precise sense, the right target — and that the posterior additionally identifies *which* predictions can affect search at all.

1.3. Contributions

1. **A reduction identity that makes in-tree supervision trivial to implement.** Conditioning (1) on a partial assignment yields a smaller BLS with the same structure (Proposition 1). One architecture, one label generator and one training loop therefore serve the root and every interior node, and a depth- d training state is stored as an ordinary instance on $n - d$ variables.
2. **A partition of branching decisions by whether they can cost anything.** The exact marginals split a node’s free variables into DET (all surviving solutions agree; branching the other way empties the subtree) and OPEN (solutions disagree; either branch reaches a solution). Only DET errors cost a backtrack. We measure the composition: 6.3% DET at the root, rising to 87.9% at depth 8 (Sections 4.3, 4.4). This locates useful prediction inside the tree rather than at the root and explains why root-level accuracy is a poor objective.
3. **A cost characterization of the learned heuristic against the sound procedure it approximates.** The network needs one relaxation and one forward pass; LP-probing needs one relaxation per variable. Measured with a warm-started backend on both sides, the ratio scales from $1.7\times$ at $n = 25$ to $17.8\times$ at $n = 150$ (Section 4.5). We report this as a scaling law rather than a single figure, and note that at the sizes where our search operates the advantage is modest and probing has the additional merit of soundness.
4. **End-to-end evaluation under matched wall-clock budgets.** At 21×60 , replicated over three seeds on a warm-started search loop, the learned guidance is $2.6\text{--}3.0\times$ faster in median than LP-based branching (faster on 23/30 instances in every seed, sign test $p = 0.005$) and expands $9.5\times$ fewer nodes; both strategies solve 30/30 within 600s (Section 4.6). In the deployed cascade both sound strategies close every residual instance, the learned one $3.0\times$ faster (40/50 wins pooled

75 over seeds, $p = 2 \times 10^{-5}$), while random branching closes none of the
76 50 residual instances (Section 4.10).
77 **5. Size transfer with solution multiplicity controlled.** Holding $|\mathcal{S}|$
78 comparable across sizes, a network trained only at 10×25 matches one
79 trained at the evaluation size 21×60 (faster on 15/30 instances; total
80 time differing by 0.4%), showing the learned quantity is not size-specific
81 (Section 4.8).

82 2. Related Work

83 2.1. Binary linear systems and market-split hardness

84 Problem (1) is a 0/1 integer feasibility problem and is NP-complete for
85 general A . Practical hardness, however, depends on the instance distribution.
86 Cornuéjols and Dawande [6] introduced the market-split family, in which m
87 agents must divide n items so that every agent’s budget is met exactly; with
88 $n \approx 10(m - 1)$ these instances defeat branch-and-bound at remarkably small
89 sizes. The mechanism is the integrality gap: the relaxation polytope is large
90 and its vertices are fractional, so bounding prunes little and the search must
91 branch nearly blindly. Aardal et al. [2] showed that lattice basis reduction
92 attacks the same family far more effectively than branch-and-bound, a result
93 we rely on in Section 3.2.

94 Our generator (Section 4.1) makes this mechanism explicit by maximizing
95 the spread of relaxation vertices, which enlarges $\tilde{\mathcal{S}}$ and weakens the relaxation
96 signal by construction.

97 2.2. Complete solvers

98 *Mixed-integer programming.* Branch-and-bound over the linear relaxation,
99 strengthened by presolve and cutting planes, is the standard approach. Its
100 leverage comes from pruning nodes whose relaxation is infeasible or bounded
101 away from the target, which is precisely what the market-split construction
102 removes.

103 *Constraint programming with clause learning.* CP-SAT-style solvers com-
104 bine constraint propagation with conflict-driven clause learning [12, 14] and
105 lazy clause generation [15]. Their strength on this family comes from a
106 property central to our argument: a search *manufactures* information as it
107 descends. A marginal $\Pr[x_j = 1 \mid A, \mathbf{b}]$ may be uninformative at the root
108 while $\Pr[x_j = 1 \mid A, \mathbf{b}, x_1=1, x_5=0]$ is 0 or 1. Structure invisible at the root

109 becomes deducible after commitments are made, which is why Section 3.4
110 studies the interior of the tree rather than the root.

111 *Lattice reduction.* An alternative attack embeds (1) in a lattice and searches
112 for short vectors. LLL [10] reduces a basis in polynomial time and BKZ
113 [16] generalizes it with a block size trading time against basis quality. The
114 Aardal–Hurkens–Lenstra embedding [1, 2] is constructed so that a 0/1 so-
115 lution appears directly as a short vector, making reduction a certificate-
116 producing procedure rather than a heuristic. We use it as the first stage of
117 the deployed pipeline.

118 2.3. Learning for combinatorial optimization

119 Bengio et al. [4] survey this field and frame it in terms we adopt: solvers
120 rely on hand-crafted heuristics for decisions that are “otherwise too expensive
121 to compute or mathematically not well defined,” and learning is a candidate
122 for making those decisions better. The phrase *too expensive to compute*
123 presumes the decision is computable in principle and asks only about cost,
124 which is the situation we measure in Section 4.5.

125 *Branching imitation.* The clearest success in this area is imitation of an
126 expensive expert. Strong branching scores candidates by tentatively solving
127 both children’s relaxations; it yields small trees and is too slow to run at
128 every node. Gasse et al. [7] train a bipartite graph network to imitate it and
129 recover most of the benefit at a fraction of the cost, following Khalil et al.
130 [9]. The gain is not new information — the expert already knew the answer
131 — but cheap access to it. Our method occupies the same structural position,
132 with two differences: the target is an exact posterior rather than an expert’s
133 scores, and the analysis identifies which of the expert’s decisions matter.

134 *Direct satisfiability prediction.* Attempts to have a network decide satisfia-
135 bility end-to-end have a weaker record. NeuroSAT [17] classifies satisfiability
136 by message passing over a literal–clause graph and succeeds on small ran-
137 dom instances, but Li et al. [11] report substantial accuracy collapse under
138 size transfer. Chen et al. [5] prove that standard message-passing networks
139 cannot separate certain feasible/infeasible MILP pairs at all, since the pairs
140 are indistinguishable in the Weisfeiler–Lehman hierarchy that bounds such
141 architectures [19]. We therefore do not ask the network for a feasibility ver-
142 dict; it supplies branching guidance inside a complete search that remains
143 responsible for soundness.

144 *Graph representations..* We use graph attention [18] over the bipartite variable–
 145 constraint graph, the standard encoding for MILP in this literature [7].

146 2.4. Amortized optimization

147 The distinction organizing our results predates its use in combinatorial
 148 optimization. Gershman and Goodman [8] introduced *amortized inference*:
 149 when many related queries must be answered, solving each from scratch is
 150 wasteful and the question becomes how to reuse computation across queries.
 151 Amos [3] develops the same idea as *amortized optimization* — using learning
 152 to predict solutions by exploiting shared structure across similar instances
 153 — and surveys its appearance in variational inference, meta-learning and
 154 control. Millidge [13] states the trade-off in the form most relevant here:
 155 *direct* optimization spends computation on the instance at hand and pays
 156 that cost at every invocation, whereas *amortized* optimization converts the
 157 problem into supervised learning with cheap inference but requires a corpus
 158 of solved instances up front and is bounded by the approximator’s gener-
 159 alization; the productive arrangement is hybrid, with amortized predictions
 160 steering a direct optimizer.

161 Our system instantiates that hybrid precisely. The direct method is LP-
 162 probing, sound and costly. The amortized method is the network, cheap and
 163 unsound. The hybrid rule is that the network steers branching while probing
 164 and propagation retain exclusive authority over variable fixing (Section 3.7).

165 3. Proposed Method

166 3.1. Setting and notation

167 For a partial assignment fixing variables $F \subseteq \{1, \dots, n\}$ to values $\mathbf{v} \in$
 168 $\{0, 1\}^{|F|}$, write

$$\mathcal{S}(F, \mathbf{v}) = \{\mathbf{x} \in \mathcal{S} : \mathbf{x}_F = \mathbf{v}\} \quad (3)$$

169 for the solutions consistent with it, and let $K = \{1, \dots, n\} \setminus F$ index the free
 170 variables. The *conditional marginal* of a free variable $j \in K$ is

$$p_j(F, \mathbf{v}) = \frac{|\{\mathbf{x} \in \mathcal{S}(F, \mathbf{v}) : x_j = 1\}|}{|\mathcal{S}(F, \mathbf{v})|}. \quad (4)$$

171 *3.2. System overview*

172 The deployed solver is a cascade of a lattice stage and a guided complete
173 search:

$$\text{propagation} \rightarrow \text{LP infeasibility rule} \rightarrow \text{kernel pump} \rightarrow \text{AHL/BKZ} \rightarrow \text{guided backtracking search} \quad (5)$$

174 The first three stages are cheap and sound. AHL/BKZ attempts a certificate-
175 producing lattice reduction with a size-dependent block parameter; when
176 it succeeds it returns a verified solution outright. The learned component
177 appears only in the final stage, and only in its branching decisions. This
178 placement matters for the evaluation: because the lattice stage is independent
179 of the branching heuristic, every heuristic under comparison receives exactly
180 the same residual set of instances, so conditioning on that residual describes
181 the algorithm rather than selecting favourable cases.

182 *3.3. Conditioning is reduction*

183 **Proposition 1.** *Let A_F and A_K denote the column submatrices of A indexed*
184 *by F and K . Then*

$$\mathcal{S}(F, \mathbf{v}) \cong \{\mathbf{y} \in \{0, 1\}^{|K|} : A_K \mathbf{y} = \mathbf{b} - A_F \mathbf{v}\}, \quad (6)$$

185 *the bijection being $\mathbf{x} \mapsto \mathbf{x}_K$. Consequently the conditional marginals (4)*
186 *of the original instance are the ordinary marginals of the reduced instance*
187 *$(A_K, \mathbf{b} - A_F \mathbf{v})$.*

188 *Proof.* $A\mathbf{x} = A_F \mathbf{x}_F + A_K \mathbf{x}_K$. Fixing $\mathbf{x}_F = \mathbf{v}$ gives $A\mathbf{x} = \mathbf{b} \iff A_K \mathbf{x}_K =$
189 $\mathbf{b} - A_F \mathbf{v}$, and $\mathbf{x} \mapsto \mathbf{x}_K$ is a bijection between the two sets since $\mathbf{x}_F$ is deter-
190 mined. The marginal statement follows by counting. $\square$

191 The identity is elementary but it is what makes in-tree supervision practi-
192 cal. A depth- d search node is not a special object requiring its own encoding;
193 it is an instance of the same problem on $n - d$ variables. One network, one
194 feature extractor and one label generator therefore cover the whole tree, and
195 training data at depth d is produced by substitution rather than by instru-
196 menting a solver.

197 Two implementation details preserve correctness. Rows that become all-
198 zero after substitution carry no constraint and are removed; retaining them
199 causes linear-programming backends to reject the model. And the free bound
200 condition $0 \leq \mathbf{b} - A_F \mathbf{v} \leq A_K \mathbf{1}$ is checked before any relaxation is solved,
201 since it detects a large fraction of dead nodes at negligible cost.

202 3.4. Which predictions can cost anything

203 At a node with surviving solution set $\mathcal{S}(F, \mathbf{v}) \neq \emptyset$, partition the free
204 variables by their conditional marginal:

$$\text{DET} = \{j \in K : p_j \in \{0, 1\}\}, \quad \text{OPEN} = \{j \in K : 0 < p_j < 1\}. \quad (7)$$

205 The distinction is operational rather than statistical:

- 206 • For $j \in \text{OPEN}$, both children of a branch on x_j contain solutions. Either
207 choice keeps a solution reachable, so *no branching decision on a OPEN*
208 *variable can cost anything*.
- 209 • For $j \in \text{DET}$, one child is empty. Branching against the forced value
210 guarantees that the subtree is exhausted without a solution, costing a
211 full backtrack.

212 This gives the conditional marginal its operational reading. It is not a
213 “probability of being correct” about some designated solution, but the *frac-*
214 *tion of the surviving solution set retained* by an assignment; and the only
215 errors that matter are those on variables where that fraction is 0 or 1.

216 Two consequences follow, both of which the experiments confirm. First,
217 an aggregate accuracy over all free variables is a poor objective, because it
218 mixes decisions that can cost a backtrack with decisions that cannot; the
219 relevant quantity is accuracy on DET. Second, since $|\mathcal{S}(F, \mathbf{v})|$ shrinks as the
220 search descends, the DET share grows with depth. This alone does not say
221 *where* a learned predictor is useful, because a DET variable that the relaxation
222 already rounds correctly needs no predictor. The useful region is where DET
223 variables are plentiful *and* the relaxation is weak, and Section 4.9 shows that
224 this is governed by constraint density $m/|K|$ rather than by depth as such:
225 on small instances the two coincide, on larger ones they do not.

226 3.5. Exact conditional marginals as training targets

227 For instances small enough to enumerate, $\mathcal{S}$ is obtained exactly with a
228 constraint solver in all-solutions mode, and (4) is computed by counting.
229 Training minimizes the cross-entropy to the resulting Bernoulli targets,

$$\mathcal{L} = -\frac{1}{|K|} \sum_{j \in K} \left[p_j \log \hat{p}_j + (1 - p_j) \log(1 - \hat{p}_j) \right], \quad (8)$$

230 which is minimized exactly at $\hat{p}_j = p_j$. Note that the same loss with a planted
 231 solution in place of p_j — the conventional choice — is a noisy surrogate
 232 whenever $|\mathcal{S}| > 1$, since the planted solution is one draw from the posterior;
 233 on our instances a Bayes-optimal predictor agrees with it only about two
 234 thirds of the time.

235 *Enumeration correctness.* A subtlety here is easy to get wrong and silently
 236 corrupts every label derived from it. Constraint solvers report distinct sta-
 237 tuses for “search exhausted” and “time limit reached with solutions already
 238 collected.” Only the former certifies that $\mathcal{S}$ is complete; accepting the latter
 239 substitutes a truncated solution set, which biases every marginal computed
 240 from it. In our implementation only **OPTIMAL** and **INFEASIBLE** are accepted
 241 and timed-out instances are discarded rather than averaged in. A useful
 242 diagnostic is that mean enumeration time equal to the time limit indicates
 243 truncated instances being counted as complete.

244 *Sampling reachable states.* A depth- d training state is built by drawing d
 245 uniformly from $\{0, \dots, 12\}$, taking a prefix of an LP-confidence variable or-
 246 dering, and fixing those variables to the values of a randomly chosen *actual*
 247 *solution*. Fixing arbitrary values would produce states with $\mathcal{S}(F, \mathbf{v}) = \emptyset$; such
 248 states are dead nodes that propagation prunes immediately and on which the
 249 conditional marginal is undefined, so they are not states a branching heuris-
 250 tic is ever asked about. The depth range is a design choice we did not tune;
 251 on 10×25 instances it yields training states with 13–25 free variables, and
 252 Section 4.9 examines what happens when the deployed search queries states
 253 outside that range, which at 21×60 is most of them.

254 3.6. Network

255 An instance is encoded as a bipartite graph with n variable nodes, m
 256 constraint nodes, and an edge (j, i) whenever $a_{ij} = 1$. Let $\tilde{\mathbf{x}}$ be the relaxation
 257 solution of the current (reduced) instance and $r_i = \sum_j a_{ij}$ the row sums.
 258 Node features are

$$\text{variable } j : (\tilde{x}_j, \deg(j)/m, |\tilde{x}_j - \text{round}(\tilde{x}_j)|), \quad (9)$$

$$\text{constraint } i : (b_i/r_i, r_i/n, (b_i - A_i\tilde{\mathbf{x}})/r_i), \quad (10)$$

259 i.e. relaxation value, column degree and fractionality for variables; tight-
 260 ness, scaled row size and relaxation residual for constraints. All are size-
 261 normalized, which is what permits a single parameter set to apply at any
 262 (m, n) .

Both node types are embedded to $h = 64$ dimensions by a linear map and refined by $L = 4$ rounds of bidirectional message passing with graph attention [18]:

$$\mathbf{h}^{\text{con}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{var}} \rightarrow \mathbf{h}^{\text{con}})) + \mathbf{h}^{\text{con}}), \quad (11)$$

$$\mathbf{h}^{\text{var}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{con}} \rightarrow \mathbf{h}^{\text{var}})) + \mathbf{h}^{\text{var}}), \quad (12)$$

followed by a two-layer head emitting one logit per variable, $\hat{p}_j = \sigma(z_j)$. The model has 21,761 parameters.

The architecture is deliberately small and single-headed. Its only output is the quantity the objective (8) defines; there are no auxiliary value or feasibility heads, because the search draws soundness from its symbolic components and asks the network for nothing else.

3.7. Inference: guidance, not commitment

The network is consulted once per node. Branching selects

$$j^* = \arg \max_{j \in K} |\hat{p}_j - \tfrac{1}{2}|, \quad \text{first value} = \mathbb{K}[\hat{p}_{j^*} \geq \tfrac{1}{2}], \quad (13)$$

that is, the variable the network is most confident about, tried at its predicted value. The search remains complete and a wrong prediction costs backtracking only.

Variables are *fixed* only by sound deduction. Two mechanisms are available: constraint propagation, and LP-probing, which fixes x_j when assigning the opposite value makes even the relaxation infeasible. The division of labour follows directly from the measurements in Section 4.5: where a wrong answer costs time, the cheap unsound predictor is preferable; where a wrong answer costs correctness, the expensive sound procedure is required and cannot be replaced.

Algorithm 1 states the resulting search.

3.8. Cost model

The comparison that motivates the design is per-node cost for a whole-instance verdict, and its structure is asymptotic rather than a constant factor. Let n_c be the number of free variables at a node. The network solves *one* relaxation to build its features and performs *one* forward pass, so its LP count is $O(1)$ in n_c . LP-probing solves one relaxation per candidate variable, $O(n_c)$. The ratio between them therefore grows linearly in instance size, and

Algorithm 1 Guided backtracking search for a BLS

Input: instance $(A, \mathbf{b})$, guidance model f_θ , time budget T
stack $\leftarrow [(A, \mathbf{b})]$
while stack $\neq \emptyset$ **and** elapsed $< T$:
 $(A', \mathbf{b}') \leftarrow \text{pop}(\text{stack})$
 if $\mathbf{b}' < \mathbf{0}$ **or** $\mathbf{b}' > A'\mathbf{1}$: **continue** // *free bound check*
 drop all-zero rows of A' ; **if** none remain: **return** any completion
 propagate; **if** contradiction: **continue**
 if propagation determined a set D of variables:
 push reduce($A', \mathbf{b}', D$); **continue** // *Proposition 1*
 if all variables assigned: **return** solution if verified, else **continue**
 $\hat{\mathbf{p}} \leftarrow f_\theta(A', \mathbf{b}')$ // *one forward pass, all variables*
 $j^* \leftarrow \arg \max_j |\hat{p}_j - 1/2|$; $v \leftarrow \mathbb{K}[\hat{p}_{j^*} \geq 1/2]$
 push reduce($A', \mathbf{b}', \{j^* = 1-v\}$), then reduce($A', \mathbf{b}', \{j^* = v\}$)
return unresolved

292 any single number quoted for it is meaningful only together with the n_c at
293 which it was measured.

294 This also means the comparison is sensitive to how the LP backend is
295 implemented, in a way that cuts against the learned method. A backend
296 that rebuilds the model for every call inflates probing's cost by its full $O(n_c)$
297 factor while charging the network only once, exaggerating the advantage. A
298 correctly engineered branch-and-bound keeps one relaxation and re-optimizes
299 from the parent basis after a bound change, which is an order of magnitude
300 cheaper per probe. Section 4.5 measures both methods against a warm-
301 started backend for this reason, and reports the ratio as a function of n_c
302 rather than as a single figure.

303 4. Experiments

304 4.1. Instances and control of solution multiplicity

305 Instances are generated by the procedure GENHARD(m, n, K): draw a
306 random 0/1 matrix A of density 1/2; plant $K = 20$ candidate solutions
307 $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(K)}$ with $\mathbf{b}^{(k)} = A\mathbf{x}^{(k)}$; retain the one maximizing *vertex spread*, the
308 mean pairwise L_2 distance among relaxation vertices obtained from random
309 objective directions. Maximizing vertex spread enlarges $\tilde{\mathcal{S}}$ directly and is

the mechanism that makes market-split instances hard for relaxation-based methods (Section 2). Uniqueness of the planted solution is not enforced.

Why $|\mathcal{S}|$ must be controlled across sizes.. Solution multiplicity changes the character of the prediction task, not only its difficulty. When $|\mathcal{S}| = 1$ every free variable is DET and predicting well is tantamount to solving; when $|\mathcal{S}|$ is large most variables are OPEN and the task is to estimate a diffuse posterior. Comparing sizes without controlling $|\mathcal{S}|$ therefore confounds size with task identity. Since $|\mathcal{S}|$ falls as m rises at fixed n , we select m per size to match it.

Size	m	m/n	median $ \mathcal{S} $	root ceiling	conditional ceiling
10×25	10	0.40	23	70.4%	87.0%
18×50	18	0.36	19	71.3%	87.2%
21×60	21	0.35	10	—	—

Table 1: Sizes used, with solution multiplicity matched. Ceilings (the Bayes-optimal per-variable accuracies of Section 4.3) agree to within one point between the two training sizes, so size is the variable that changes.

Limits of the control.. Matching $|\mathcal{S}|$ forces m/n to vary ($0.40 \rightarrow 0.35$); at fixed n the two cannot be held simultaneously and we prioritize $|\mathcal{S}|$ because it determines task identity. The control is also granular: at $n = 60$ the median $|\mathcal{S}|$ steps $42 \rightarrow 12 \rightarrow 2$ across $m = 20, 21, 22$, so $m = 21$ is the best available integer and the realized evaluation set has median $|\mathcal{S}| = 10$ with 27/30 verified.

Where enumeration stops.. At $n = 70$ no enumeration completed within 120s at the m giving the target multiplicity (0/10). At $n = 100$ the obstruction is more basic: a constraint solver finds *zero* solutions within 20s at $m \in \{28, 32, 36, 40\}$ despite a planted one. At that size the bottleneck is finding any solution rather than choosing branches well, so a branching comparison there would not be informative. We therefore evaluate at $n \leq 60$ and report this boundary as a limitation.

4.2. Implementation and protocol

Software.. Relaxations used for training features and for the original search loop are solved with SCIP via PySCIPOpt; the warm-started relaxations

of Sections 4.5 and 4.6 use Gurobi 13 (dual simplex, bounds changed in place), which requires a licence — HiGHS exposes an equivalent incremental interface and should serve as a free substitute, though we have not verified the timings with it. Enumeration and the CP-SAT baseline use OR-Tools; lattice reduction uses fpylll (LLL and BKZ). The network is implemented in PyTorch with PyTorch Geometric (GATConv). One property of the relaxation deserves note: it carries no objective, so its solution is an arbitrary vertex of the relaxation polytope and different LP codes return different vertices. Both branching arms consume the same vertex within an experiment, so comparisons are fair, but absolute node counts differ between the SCIP- and Gurobi-backed runs.

Training.. Adam, learning rate 10^{-3} , 20 epochs over 10,000 in-tree states, batch size 1 (instances vary in size), gradient-norm clipping at 1.0, loss (8) with exact marginal targets. Training states are drawn at depths 0–12 uniformly, 6 states per source instance, from a pool of 1,760 instances; evaluation uses 440 instances from a disjoint pool. Training takes approximately 20 minutes on one CPU core. *No hyperparameter was tuned*: width, depth, learning rate, epoch count and the training depth range are the values first tried, and there is no validation split. Reported figures are therefore not contaminated by selection on the test set, but neither is any of these settings claimed to be optimal.

Threading.. All timing-sensitive components run single-threaded (`torch.set_num_threads(1)`). On graphs of this size multi-threaded BLAS is counter-productive: we measure 11.3ms per forward pass at 8 threads against 0.9ms at one.

Search evaluation.. Each arm is run on identical instances with a hard per-instance wall-clock budget enforced by subprocess termination, since lattice reduction executes inside a blocking native call that in-process timers cannot interrupt. Instances run at concurrency 6 on a 20-core machine. This low concurrency is deliberate: wall-clock-budgeted comparisons are sensitive to CPU contention, and at higher concurrency we have observed instances solvable in 14–15s standalone exceeding a 20s cap.

Statistics.. Solve-rate differences between arms on the same instances are tested with McNemar’s exact test; per-instance speed differences with a two-sided sign test. We report both rather than a single aggregate because they

370 can have different power. The main comparison at 21×60 is replicated over
 371 three independent seeds and we report the spread; smaller-size and transfer
 372 results are single-run and marked as such. Seeds are set explicitly: an earlier
 373 version derived them from `hash()` of the instance filename, which Python
 374 randomizes per process, so those runs had no recorded provenance.

375 4.3. Prediction quality against the Bayes ceiling

376 Because $\mathcal{S}$ is enumerated exactly, the Bayes-optimal per-variable accuracy
 377 is computable:

$$\text{ceil} = \frac{1}{|K|} \sum_{j \in K} \max(p_j, 1 - p_j), \quad (14)$$

378 attained by answering the majority value of the surviving solutions. No
 379 deterministic predictor of $\mathbf{x}^*$ from $(A, \mathbf{b})$ can exceed it. Table 2 evaluates
 380 predictors against it on 400 held-out 10×25 instances with $\mathcal{S}$ fully enumer-
 381 ated.

Predictor	all variables	top-3	L_1 to true marginal
Bayes ceiling	70.8%	93.2%	0.0000
Proposed network	66.7%	86.6%	0.1216
LP relaxation rounding	60.9%	65.8%	—

Table 2: Root-level prediction against the exact posterior on the 182 root states of the held-out 10×25 evaluation pool. The network is the same checkpoint used in every search experiment below. “top-3” restricts to the three variables the predictor is most confident about.

382 The network comes within 4.1 points of the ceiling overall and 6.6 on the
 383 top-3 subset, and it recovers most of the gap between relaxation rounding
 384 and the ceiling ($60.9 \rightarrow 66.7$ against a ceiling of 70.8). Two preliminary
 385 experiments on an earlier pool of 10×25 instances, reported for completeness
 386 but not from the checkpoint evaluated here, bear on design choices: a 2.11M-
 387 parameter network with four output heads (selection, assignment, value and
 388 feasibility) scored 60.7% on root states, statistically indistinguishable from
 389 relaxation rounding at 60.6%, so capacity is not the binding constraint and
 390 a single-purpose head is preferable; and training the single-head architecture
 391 on exact marginals rather than the planted solution improved L_1 to the
 392 posterior from 0.1261 to 0.0977 while leaving argmax accuracy unchanged,

393 which matters because the analysis of Section 3.4 consumes the marginal
 394 rather than its argmax.

395 *What the aggregate conceals..* Table 3 decomposes the same root variables
 396 by the partition of Section 3.4.

Root variables	count	share	ceiling on subset
DET (branching against it empties the subtree)	38	6.3%	100.0%
OPEN (either branch reaches a solution)	562	93.7%	67.2%
all variables	600	100%	69.3%

Table 3: Root-level decomposition, 24 instances of 10×25 with $\mathcal{S}$ enumerated. The aggregate is 94% composed of variables on which no branching decision can cost a backtrack.

397 At the root, 93.7% of variables are OPEN. The aggregate ceiling of 69.9%
 398 is therefore dominated by predictions that cannot affect the search, and on
 399 the 6.3% that can, the information is already complete (100%). Root-level
 400 accuracy is consequently a weak objective for a branching heuristic, and the
 401 operative question is not whether the information exists but what detects it
 402 — taken up next.

403 4.4. Depth, and where the residual lies

404 Using Proposition 1 we measure, at each depth along a prefix drawn
 405 from an actual solution, the conditional ceiling and the fraction of remaining
 406 variables that propagation forces (Table 4).

depth	surviving $ \mathcal{S} $	conditional ceiling	proposed network	LP rounding
0	25.3	70.8%	66.7%	60.9%
4	4.2	82.3%	72.8%	68.5%
7	1.8	91.5%	81.2%	77.8%
8	1.4	94.4%	84.3%	80.9%
12	1.1	98.8%	96.8%	95.5%
all depths	—	87.2%	80.0%	76.5%

Table 4: Conditional information against depth on the held-out in-tree states of the 10×25 evaluation pool ($n = 25$), LP-confidence variable ordering. Same checkpoint as Table 2.

407 The consequential movement is in composition rather than level: the
 408 DET share rises from 6.3% at the root to 87.9% at depth 8 (Table 6). At the

409 root almost every branching decision is inconsequential because both children
 410 contain solutions; by depth 8 almost every decision empties a subtree when
 411 wrong. The network’s margin over relaxation rounding is 3–6 points through-
 412 out and does not vanish at depth, which is the first indication that what it
 413 captures is not simply depth-dependent; Section 4.9 makes this precise.

414 *Does in-tree supervision matter, controlling for everything else?*. A compari-
 415 son between a root-trained and an in-tree-trained network is only informative
 416 if nothing else differs. Table 5 holds the instance pool, the label type (exact
 417 marginals), the architecture, the optimizer and the number of training states
 418 (1,760 each) fixed, and varies only whether the states are root states or in-
 419 tree states drawn at depths 0–12. Both are evaluated on the same held-out
 420 in-tree states.

depth	conditional ceiling	root-trained	in-tree-trained	LP rounding
0	70.7%	67.8%	66.5%	60.9%
4	82.1%	71.4%	72.4%	68.3%
6	89.7%	74.6%	78.2%	74.0%
8	94.4%	77.4%	84.2%	81.3%
10	97.1%	81.7%	91.4%	89.4%
12	98.8%	85.8%	96.5%	95.3%
all depths	87.2%	75.4%	79.8%	76.4%

Table 5: Controlled comparison of training-state distribution. Same pool, labels, architecture, optimizer and state count; only the depth at which states are sampled differs. Evaluated on held-out in-tree states of the 10×25 pool.

421 In-tree supervision is worth 4.4 points overall. The more telling row is the
 422 root-trained network itself: at 75.4% it is *below* relaxation rounding on in-
 423 tree states, so a network trained on root states does not merely underperform
 424 in the tree, it is worse than the classical heuristic it is meant to replace. The
 425 shift from root to interior states is not a distribution the root-trained network
 426 generalizes across, and in-tree training is what makes the learned guidance
 427 better than the baseline at all.

428 Table 6 decomposes the residual gap at these depths and asks what each
 429 method detects.

430 The gap lies entirely on DET variables: at depth 7, $0.80 \times 11.2 \approx 9$ points,
 431 matching the measured 10.3-point shortfall against the conditional ceiling.

depth	% DET	network on DET	propagation	LP integrality	LP-probing	network on OPEN
5	61.9%	89.7%	1.2%	67.9%	51.6%	54.4%
6	74.2%	88.2%	1.6%	70.4%	68.0%	52.5%
7	79.9%	88.8%	3.7%	73.3%	72.8%	51.2%
8	87.9%	89.4%	5.3%	76.9%	79.8%	53.8%

Table 6: Residual decomposition on the same states as Table 4. Columns 4–6 are recall on the ground-truth DET set (LP-probing measured with the warm-started backend of Section 4.5); columns 3 and 7 are the network’s argmax accuracy on each subset.

432 Performance near 50% on OPEN variables is correct behaviour rather than
433 a defect, since those variables are genuinely undetermined. The striking
434 figure is that unit propagation detects only 1.2–5.3% of logically determined
435 variables: the information is present and derivable but not reachable by the
436 cheapest sound rule. LP-probing recovers 52–80% and LP integrality 68–
437 77%.

438 4.5. Cost per node

439 Both methods are charged everything they must pay to reach a whole-
440 instance verdict: the network one relaxation for its features plus one forward
441 pass, probing one relaxation per free variable. Both use the same warm-
442 started backend, a single model whose variable bounds are changed in place
443 and re-optimized from the parent basis (Gurobi dual simplex), which is how a
444 branch-and-bound implementation reuses its relaxation. Timings are single-
445 threaded.

446 The ratio behaves as the cost model predicts: it is negligible at $n = 25$
447 and reaches $17.8\times$ at $n = 150$, tracking the $O(n_c)$ versus $O(1)$ difference
448 in relaxation count. Two consequences follow, and they point in opposite
449 directions.

450 *The advantage is real but asymptotic, and small where we can search..* At the
451 sizes where our search actually operates ($n = 25$ – 60) the ratio is 1.3 – $7.2\times$.
452 At the depths where prediction matters most (5–8 of a 10×25 instance,
453 $n_c = 17$ – 20) it is 1.3 – $1.7\times$. A practitioner choosing between the two at
454 that scale should prefer probing, which is *sound*: where its test fires the
455 conclusion is a proof, whereas the network never offers a guarantee. The
456 learned alternative becomes compelling only at $n \gtrsim 100$, and Section 4.1
457 shows that search itself fails there for every strategy we tried.

Instances	n	network	LP-probing	ratio	probing recall
10×25	25	3.22 ms	5.60 ms	$1.7\times$	—
18×50	50	2.33 ms	11.84 ms	$5.1\times$	—
21×60	60	2.57 ms	18.39 ms	$7.2\times$	—
40×100	100	4.02 ms	62.46 ms	$15.5\times$	—
60×150	150	4.45 ms	79.06 ms	$17.8\times$	—
<i>in-tree states of 10×25 instances, by depth:</i>					
depth 5	20	2.44 ms	4.02 ms	$1.6\times$	51.6%
depth 6	19	2.39 ms	3.98 ms	$1.7\times$	68.0%
depth 7	18	2.35 ms	3.43 ms	$1.5\times$	72.8%
depth 8	17	2.34 ms	3.13 ms	$1.3\times$	79.8%

Table 7: Per-node cost for a whole-instance verdict, warm-started LP backend for both methods, single-threaded. The network’s accuracy on DET variables at depths 5–8 is 85.9–92.7% against probing’s recall in the last column.

458 *The measurement is sensitive to baseline engineering.* An earlier version of
459 this comparison used a backend that rebuilt the LP model on every call
460 and, additionally, excluded the network’s own feature relaxation from its
461 timer. That combination reported 19–23 $\times$ at depths 5–8, where the correctly
462 measured value is 1.3–1.7 $\times$ — an overstatement of more than an order of
463 magnitude, arising entirely from how the two sides were instrumented rather
464 than from what they compute. We report the corrected figures and note
465 the general hazard: in any cost comparison against a classical procedure, the
466 classical side must be implemented as a practitioner would implement it, and
467 the learned side must be charged for its own preprocessing.

468 4.6. Search performance

469 We compare branching strategies under matched wall-clock budgets with
470 the lattice stage disabled, isolating branching quality. LP selects the variable
471 with the most integral relaxation value and tries that value first; MODEL uses
472 the proposed network. Both run in the same loop: one warm-started relax-
473 ation per node (Section 4.2), identical propagation, identical value ordering,
474 so the arms differ only in which variable they branch on and which value
475 they try first. Each configuration is run three times with different seeds; the
476 depth-first search itself is deterministic given the relaxation vertex, so seeds
477 change timing but not node counts.

Size (budget)	strategy	solve rate	median nodes	median time	nodes/s
10×25 (60 s)	LP	$100 \pm 0.0\%$	46	0.03 ± 0.00 s	1,767
	MODEL	$100 \pm 0.0\%$	24	0.07 ± 0.01 s	376
18×50 (300 s)	LP	$100 \pm 0.0\%$	4,224	2.43 ± 0.33 s	2,066
	MODEL	$100 \pm 0.0\%$	1,085	1.82 ± 0.37 s	731
21×60 (600 s)	LP	$100 \pm 0.0\%$	82,366	51.5 ± 2.0 s	1,627
	MODEL	$100 \pm 0.0\%$	8,657	14.6 ± 0.7 s	619

Table 8: Branching strategy comparison on the warm-started loop, lattice stage disabled, 30 instances per cell, mean $\pm$ standard deviation over three seeds of the per-seed median. Node counts are seed-invariant.

Three points. First, the learned guidance produces a smaller tree at every size — $2.0\times$, $3.9\times$ and $9.5\times$ fewer nodes than LP — and the ratio grows with size, as the analysis of Section 4.9 predicts: the states where the network separates from relaxation rounding are the large, weakly constrained ones near the top of a big tree, and 10×25 has few of them. Second, a smaller tree buys time only once the tree is large enough to pay for the forward pass. At 10×25 the MODEL arm is *slower* (0.34 – $0.43\times$, faster on at most $1/30$ instances), because a 24-node tree costs less than the network evaluations that shrink it; its per-node cost is 2.6 – $4.7\times$ that of LP at every size, which is the forward pass of Section 4.5. At 18×50 the median per-instance ratio is 1.30 – $1.37\times$ in the network’s favour but not significant (faster on $18/30$ in every seed, sign test $p = 0.36$). Third, at 21×60 the advantage is clear and replicates: the median per-instance ratio is $2.57\times$, $3.03\times$ and $2.96\times$ across seeds, the network is faster on exactly $23/30$ instances in each ($p = 0.005$), and both arms solve every instance within budget.

Two further strategies were measured on an earlier version of this loop that rebuilt the relaxation at every node. RANDOM branching solved $0/30$ at 20×50 within 300 s, and Section 4.10 shows it closes none of the 21×60 residual instances on the present loop either. LP-PROBE, which probes every free variable and soundly fixes the forced ones before branching as LP, produced by far the smallest trees ($51\times$ fewer nodes than LP at 20×50) and was the slowest arm, processing 6 nodes/s against 376: node count and wall-clock rank the methods in opposite orders, and a tree-size convention would select the worst of them. We have not repeated LP-PROBE on the warm-started loop; Section 4.5 measures its per-node cost there directly.

503 We also record what the rebuilt-relaxation loop reported for the 21×60
504 comparison, because an earlier draft of this paper rested on it: 30/30 against
505 25/30 solved, and a $3.29\times$ median speedup. Warm-starting the relaxation
506 removed the solve-rate difference entirely — the five instances LP failed within
507 600 s it now closes in 65–270 s — and reduced the speedup. The solve-rate
508 advantage was an artifact of a slow backend that penalized the arm expanding
509 more nodes, and we do not claim it.

510 4.7. Comparison against complete solvers

511 Table 8 compares branching strategies inside our own search loop, which
512 measures branching quality but says nothing about how the system stands
513 against an off-the-shelf solver. That comparison is reported here, on the same
514 instances, single-threaded, under the same budget.

Size	CP-SAT	SCIP	proposed (MODEL)	CP-SAT advantage
10×25	0.002 s	0.009 s	0.07 s	$35\times$
18×50	0.046 s	0.385 s	1.82 s	$40\times$
21×60	1.159 s	3.687 s	14.6 s	$13\times$

Table 9: Median solve time; all methods solve 30/30 at every size. The proposed column is the MODEL arm of Table 8 (mean of per-seed medians). The proposed system does not beat a complete solver at any size tested, and is outperformed by SCIP, whose LP code it uses for training features.

515 The gap is large and we state it plainly: *the contribution of this paper is*
516 *not a solver that competes with CP-SAT*. It is worth asking where the gap
517 comes from, because the answer bears on what the branching heuristic is
518 actually doing. Table 10 decomposes it at 21×60 into tree size and per-node
519 throughput, using CP-SAT’s own branch counter.

	CP-SAT	proposed	ratio
nodes (median)	18,827 branches	8,657	ours $2.2\times$ fewer
throughput	19,782 nodes/s	619 nodes/s	CP-SAT $32\times$ faster
total time (median)	1.065 s	14.6 s	$14\times$

Table 10: Decomposition of the 21×60 gap. CP-SAT additionally learns a median of 5,514 conflict clauses per instance.

520 The learned guidance produces a *smaller* search tree than CP-SAT —
521 $2.2\times$ fewer nodes despite CP-SAT’s clause learning — and loses entirely on
522 per-node cost. Where that cost goes can be measured. Before the loop was
523 warm-started it processed 254 nodes/s, spending 4.34 ms per node rebuilding
524 and solving the relaxation; warm-starting the same LP brings that to 0.08 ms
525 and raises throughput to 619 nodes/s, which is the loop reported throughout
526 Section 4.6. Of the 1.6 ms that remain per node, the network’s forward pass
527 is 1.88 ms at the root and less at deeper nodes where the reduced graph is
528 smaller, and propagation and bookkeeping are under 0.1 ms: the forward
529 pass now accounts for over 90% of per-node cost, and we have not reduced
530 it. We therefore do not claim that the gap is an implementation artifact.
531 What the node counts establish is narrower: the deficit is not in branching
532 quality. Closing it would require at minimum a compiled inference path and,
533 as the next paragraph notes, algorithmic components our search lacks.

534 Two honest caveats attach to this reading. Tree sizes are not perfectly
535 commensurable — CP-SAT’s “branches” include restarts and conflict-driven
536 jumps, and each of its nodes performs more propagation work than ours
537 — so the $2.2\times$ should be read as an order-of-magnitude statement. And
538 a competitive implementation would need more than a faster forward pass:
539 clause learning and restarts are absent from our search, and the last two are
540 what let CP-SAT scale past the sizes where we stop.

541 4.8. Size transfer

542 Because the features are size-normalized and the parameters do not de-
543 pend on (m, n) , one network applies at any size. We test whether that
544 transfers in practice, with $|\mathcal{S}|$ matched as in Table 1.

Training size	Evaluation size	solve rate	median time	median nodes
10×25	10×25	100%	0.07 ± 0.01 s	24
10×25	18×50	100%	1.82 ± 0.37 s	1,085
18×50	18×50	100%	1.48 ± 0.15 s	942
10×25	21×60	100%	14.6 ± 0.7 s	8,657
18×50	21×60	100%	23.7 ± 0.7 s	14,742

Table 11: Transfer on the warm-started loop, lattice stage disabled, 30 instances per row, three seeds. Rows 2–3 and 4–5 each hold the evaluation size fixed and vary only the training size.

At the 21×60 evaluation size the network trained only at 10×25 — a $2.4 \times$ smaller problem — is not worse than the one trained at 18×50 ; if anything it is better, expanding fewer nodes on $18/30$ instances and finishing faster on $18\text{--}19/30$ in each seed, with a median per-instance time ratio of $1.24\text{--}1.31 \times$ in its favour. Neither difference is significant (sign test $p \geq 0.20$), and both solve $30/30$. At 18×50 the picture is the same: the 10×25 network is faster on $16\text{--}17/30$ instances ($p \geq 0.59$) although its median time is slightly higher, the two summaries disagreeing because the paired differences are small against instance-to-instance variation. A previous version of this comparison, on the rebuilt-relaxation loop, gave $15/30$ and a $0.92 \times$ ratio; the direction is not stable across loops and we read the result as no detectable transfer loss rather than as a transfer gain. Root-level prediction agrees: the 10×25 network scores 67.8% , 69.5% and 69.4% at 10×25 , 20×50 and 40×100 , holding its margin over relaxation rounding ($+11.0$, $+7.2$, $+4.7$ points) at every size.

This matters practically because label generation is the binding constraint. Exact marginals require enumeration, which fails beyond $n \approx 60$; transfer means a network trained where labels are obtainable can be deployed where they are not.

4.9. Where the transferred network helps

Section 4.4 located the useful prediction regime at depths 6–8 of a 10×25 instance. Whether that localization survives transfer is a separate question, and it does not: at 21×60 the search queries the network on states with 5–60 free variables, only 19.7% of which fall inside the training range of $13\text{--}25$, and 80.2% of which are larger than any training state. Table 12 evaluates the network and relaxation rounding against exact conditional marginals on 21×60 in-tree states, grouped by the number of free variables, on the subset of states that could be enumerated within 15 s.

The pattern is the reverse of what a depth-based account predicts. Inside the training range the network is indistinguishable from relaxation rounding — both are perfect and their outputs correlate at 1.000 — because at $m/|K| \geq 0.8$ the reduced instance is over-constrained and the relaxation is already integral on every DET variable. The network’s entire margin arises on states it never saw during training, at $50\text{--}60$ free variables, where the relaxation is weakest ($m/|K| \approx 0.35\text{--}0.42$) and its outputs have decorrelated from the relaxation’s. In terms of Section 3.4, this is the region where DET variables are present but the relaxation does not find them.

free vars	in training range	$m/ K $	network on DET	LP on DET	gain	corr(network, LP)
20	yes	1.05	100.0%	100.0%	+0.0	1.000
25	yes	0.84	100.0%	100.0%	+0.0	0.998
30	no	0.70	95.7%	95.0%	+0.7	0.954
40	no	0.53	75.5%	75.2%	+0.4	0.747
50	no	0.42	70.2%	65.0%	+5.3	0.664
60	no	0.35	93.3%	88.9%	+4.4	0.658

Table 12: 21×60 in-tree states by number of free variables, 9–14 enumerable states per row. Gain is the network’s margin over relaxation rounding on DET variables; the last column is the Pearson correlation between the two predictors’ outputs.

Two conclusions follow. The first corrects Section 4.4: depth was a proxy for constraint density that happened to hold on 10×25 , where descending from the root moves $m/|K|$ only from 0.40 to 0.59, and does not hold on 21×60 , where the same descent moves it from 0.35 past 1.0. The useful regime is defined by $m/|K|$, and at deployment size it sits near the *top* of the tree. The second is that the transfer result of Section 4.8 is stronger than a matched-multiplicity comparison alone shows: the network extrapolates to states outside its training range in both size and constraint density, and that extrapolation is where its contribution lives. It is also a limitation, since the training range was not chosen with this in mind (Section 4.12).

4.10. Deployment in the full pipeline

Table 8 isolates branching by disabling lattice reduction. In deployment the lattice stage runs first and the search sees only its residual. Because that stage is independent of the branching heuristic, every arm receives the identical residual set.

Size	strategy	closed by lattice	residual solved by search	median time on residual	end-to-end
10×25	both	30/30	—	—	100%
18×50	LP / MODEL	24–28/30	11/11 / 11/11	0.5–5.0 s / 0.6–2.5 s	100% / 100%
21×60	RANDOM	10–16/30	0/50	> 600 s	$44.4 \pm 8.3\%$
21×60	LP	10–16/30	50/50	52.9–65.1 s	$100 \pm 0.0\%$
21×60	MODEL	10–16/30	50/50	14.2–20.2 s	$100 \pm 0.0\%$

Table 13: Cascade results on the warm-started loop, three seeds at every size, 600s budget at 21×60 . Lattice coverage varies across seeds (16, 14, 10 of 30 at 21×60) because the lattice stage draws random column permutations, so the residual handed to the search varies with it; all arms within a seed receive the identical residual.

597 Lattice coverage falls with size — 30/30, 24–28/30, 10–16/30 — and that
 598 decline is what creates room for branching quality to matter. At 10×25
 599 and 18×50 the residual is empty or closed within seconds by either sound
 600 strategy. At 21×60 between ten and twenty instances reach the search
 601 depending on the seed, and the residual is genuinely hard: random branching
 602 closes none of the 50 residual instances pooled over seeds, so the end-to-
 603 end rate of that arm is the lattice coverage alone. Both relaxation-based
 604 and learned branching close all 50, and the difference between them is time.
 605 On the residual the learned guidance is faster on 11/14, 14/16 and 15/20
 606 instances in the three seeds (median per-instance ratio $3.48\times$, $2.99\times$, $2.38\times$;
 607 sign test $p = 0.057$, 0.004 , 0.041), 40/50 pooled with a median ratio of $3.00\times$
 608 ($p = 2.4 \times 10^{-5}$), expanding 6.9–9.5 $\times$ fewer nodes. The residual is where the
 609 branching heuristic operates in deployment, and the speedup there matches
 610 the isolated measurement of Table 8.

611 4.11. Does the graph structure contribute?

612 MarginalNet consumes six scalars per node and propagates them over
 613 the bipartite graph. With a feature set that small, a per-variable model
 614 ignoring the graph is a serious baseline: if it matches, the message passing is
 615 decoration. Table 14 compares models trained on identical targets, loss and
 616 splits, differing only in whether structure is used. The aggregated variant
 617 appends mean and max of the constraint-side features over each variable’s
 618 incident rows, which is the information one message-passing round would
 619 deliver.

Model	accuracy	L_1 to marginal	parameters
Linear, variable features only	78.1%	0.2530	4
MLP, variable features only	77.8%	0.2522	8,641
MLP + constraint aggregates	79.4%	0.2361	9,025
MarginalNet (bipartite graph)	84.1%	0.1514	21,761
Bayes ceiling	87.3%	0.0000	—

Table 14: Ablation on in-tree states of 10×25 instances, 2,000 training and 800 test states.

620 Message passing contributes: +4.7 points over the best structure-free
 621 model and a 36% reduction in L_1 , closing roughly 60% of the residual gap to
 622 the ceiling that the MLP leaves open. The improvement in L_1 is the more

623 relevant of the two, since the analysis of Section 3.4 consumes the marginal
 624 rather than its argmax.

625 The same table bounds the claim, though. A four-parameter linear model
 626 on the relaxation value alone reaches 78.1%, within 9.2 points of the ceiling;
 627 most of the available signal is in that single feature, and the graph adds a
 628 useful but not dominant increment on top of it.

629 4.12. Limitations

630 *Label generation bounds the method, not only its evaluation..* Exact marginals
 631 require enumerating $\mathcal{S}$, which does not complete beyond $n \approx 60$ on this fam-
 632 ily. Transfer permits deployment at larger sizes but not training there; ap-
 633 proximating p_j by solution sampling is the natural extension and is untested.

634 *The useful size window is bounded on both sides..* Below it, lattice reduction
 635 closes everything and branching quality is irrelevant. Above it ($n = 100$),
 636 search fails outright for every strategy, since a constraint solver finds no
 637 solution within 20 s. Our demonstration lives at $n = 60$ and we do not show
 638 the window extends further.

639 *The advantage is in time and tree size, not in solve rate..* On a warm-started
 640 loop both sound strategies solve every instance we evaluate within budget;
 641 what the learned guidance buys at 21×60 is a $2.6\text{--}3.0\times$ median speedup
 642 and $9.5\times$ fewer nodes, significant per instance in every seed. An earlier draft
 643 reported a 5–0 solve-rate advantage from a loop that rebuilt the relaxation
 644 at every node; that difference vanished with the faster backend and we do
 645 not claim it. Whether a solve-rate advantage exists at larger budgets or sizes
 646 is untested, and with 30 evaluation instances per size the power to detect
 647 one would be low in any case.

648 *Multiplicity control is approximate..* Median $|\mathcal{S}|$ is 23, 19 and 10 across the
 649 three sizes; at $n = 60$ multiplicity steps $42 \rightarrow 12 \rightarrow 2$ across consecutive inte-
 650 ger m , so finer matching is unavailable. Three of thirty evaluation instances
 651 have unverified $|\mathcal{S}|$ and are marked as such. Matching $|\mathcal{S}|$ also moves m/n
 652 from 0.40 to 0.35.

653 *Cost advantage depends on scale and on baseline engineering..* The amortiza-
 654 tion argument holds asymptotically but is worth little at the sizes where our
 655 search operates: $1.3\text{--}1.7\times$ at the depths where prediction matters, against a
 656 warm-started probing baseline that is additionally sound. It reaches $15\text{--}18\times$

only at $n \geq 100$, where search fails for every strategy we tested. We therefore cannot demonstrate the regime in which the argument is strongest.

The search implementation is not competitive.. At 21×60 CP-SAT is $13\times$ faster end to end. Our branching produces fewer nodes, so the deficit is per-node throughput ($32\times$), now dominated by the network’s forward pass; we have not shown that a competitive implementation is achievable: a compiled inference path, clause learning and restarts are all absent, and the latter two are load-bearing for CP-SAT at larger sizes.

Training and deployment distributions barely overlap.. The training range $\{0, \dots, 12\}$ was not chosen with deployment in mind; it produces states with 13–25 free variables, whereas the 21×60 search queries states with 5–60 and spends 80% of its queries above 25. The method works in spite of this rather than because of it (Section 4.9), and aligning the training range with the deployment regime is an obvious untested improvement.

Hyperparameters are untuned and there is no validation split.. See Section 4.2. The configuration is defensible as a first attempt and is not contaminated by the test set, but a reviewer is right to ask whether a tuned baseline MLP or a tuned depth range would change the margins.

Single instance family.. All results concern one generator from one application. Whether the amortization argument transfers to other constraint families is untested.

5. Conclusion

We presented a learned branching heuristic for binary linear systems together with an exact procedure for supervising it. Two elements carry the method. Conditioning a BLS on a partial assignment is identical to reducing it, so a single network trained on ordinary instances predicts at every node of the search tree without architectural change. And on instances small enough to enumerate, the exact conditional posterior both supplies the training target and identifies which predictions can affect the search at all: only variables that every surviving solution agrees on can cost a backtrack, and their share rises from 6.3% at the root to 87.9% at depth 8.

Measured inside our search loop the heuristic does what it is designed to do. It is $2.6\text{--}3.0\times$ faster in median than relaxation-based branching at 21×60

690 in each of three seeds, expands $9.5\times$ fewer nodes, and closes the residual of
691 the deployed pipeline $3.0\times$ faster, where random branching closes nothing;
692 with solution multiplicity controlled, a network trained at $2.4\times$ smaller scale
693 performs indistinguishably from one trained at the evaluation size, which
694 matters because exact supervision is available only at small sizes.

695 We are equally explicit about what the results do not establish. The sys-
696 tem does not compete with a complete solver: CP-SAT is $13\text{--}40\times$ faster at
697 every size we tested. Decomposing that gap is informative rather than excul-
698 patory — our branching yields $2.2\times$ fewer nodes and loses on throughput by
699 $32\times$, which places the deficit in a Python research implementation whose per-
700 node cost is now the network’s forward pass, and which lacks clause learning
701 and restarts — but we have not shown that closing it is achievable. Nor does
702 the cost argument carry the weight we initially assigned it. Against a prop-
703 erly warm-started probing baseline the advantage is $1.3\text{--}1.7\times$ at the depths
704 where prediction matters on our instances, and probing is sound where the
705 network is not; the ratio reaches $15\text{--}18\times$ only at $n \geq 100$, a regime in which
706 search fails for every strategy we tried. An earlier version of this compar-
707 ison, using a backend that rebuilt the LP for each probe and excluded the
708 network’s own feature solve from its timer, reported $19\text{--}23\times$ at a scale where
709 the true figure is near unity, and we record the correction because the error
710 came entirely from instrumentation rather than from what the two methods
711 compute.

712 What remains, and what we claim, is methodological. For planted-
713 solution instance families the exact conditional posterior is computable and is
714 a well-posed supervision signal for branching, where the conventional target
715 — a single planted solution — is not. States drawn from inside the search
716 tree are a better training distribution than root states, and the reduction
717 identity makes generating them a matter of substitution rather than solver
718 instrumentation. The resulting heuristic transfers across instance size when
719 solution multiplicity is held comparable. Whether this becomes practically
720 useful depends on two things we have not done: extending exact supervision
721 past $n \approx 60$, most plausibly by approximating the marginals without exhaus-
722 tive enumeration, and implementing the search well enough that a branching
723 improvement is visible at the system level.

724 Reproducibility

725 Code is organized by component under `proposed_src/`, with shared mod-
726 ules in `util/` and a `ROLES.txt` in each directory describing every file. Scripts
727 are run from their own directory. Every search number in Tables 8, 11
728 and 13 comes from one chain, `v23_ablation/run_revision2.sh`, which runs
729 the warm-started loop (`v23_warm_search.py`) over all conditions and seeds;
730 `v23_aggregate.py` produces the tables from its output.

```
731 # instances with matched |S| and exact marginals
732 cd proposed_src/v15_bayes
733 python3 v15_gen_by_m.py --m 10 --n 25 --count 2200 --K 20 \
734     --out ../../runs/data/sols_10x25.json
735
736 # in-tree training states (depths 0-12, 6 per instance)
737 cd ../v17_conditional
738 python3 v17_make_conditional_data.py \
739     --sols ../../runs/data/full_10x25_train.json \
740     --max_depth 12 --per_instance 6 \
741     --out ../../runs/data/cond_10x25_train.json
742
743 # train (about 20 minutes, single CPU core)
744 python3 v17_train_conditional.py \
745     --train ../../runs/data/cond_10x25_train.json \
746     --test  ../../runs/data/cond_10x25_test.json \
747     --n_train 10000 --n_test 2000 --epochs 20 \
748     --target soft --out ../../runs/model_n25
749
750 # every search table: warm-started loop, 3 seeds,
751 # conditions A/B/R/C/D/Crand x {lattice off, on}
752 cd ../v23_ablation
753 bash run_revision2.sh          # -> runs/v23/, chain.log
754 python3 v23_aggregate.py      # -> runs/v23/aggregate.json
```

755 All generators are seeded. The released checkpoints are 96 KB each. In-
756 stance data and weights are regenerated from the above rather than dis-
757 tributed, and the full evaluation cycle is scripted in `run_revision2.sh` under
758 `v23_ablation/`.

759 References

- 760 [1] Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a
761 system of linear Diophantine equations with lower and upper bounds on
762 the variables. *Mathematics of Operations Research*, 25(3):427–442.
- 763 [2] Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and
764 Smeltink, J. W. (2000). Market split and basis reduction: Towards a
765 solution of the Cornuéjols–Dawande instances. *INFORMS Journal on*
766 *Computing*, 12(3):192–202.
- 767 [3] Amos, B. (2023). Tutorial on amortized optimization. *Foundations and*
768 *Trends in Machine Learning*, 16(5):592–732.
- 769 [4] Bengio, Y., Lodi, A., and Prouvost, A. (2021). Machine learning for
770 combinatorial optimization: A methodological tour d’horizon. *European*
771 *Journal of Operational Research*, 290(2):405–421.
- 772 [5] Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On represent-
773 ing mixed-integer linear programs by graph neural networks. In *Inter-*
774 *national Conference on Learning Representations*.
- 775 [6] Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 pro-
776 grams. In *Integer Programming and Combinatorial Optimization*, pages
777 284–293.
- 778 [7] Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019).
779 Exact combinatorial optimization with graph convolutional neural net-
780 works. In *Advances in Neural Information Processing Systems*.
- 781 [8] Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in
782 probabilistic reasoning. In *Proceedings of the 36th Annual Meeting of*
783 *the Cognitive Science Society*, pages 517–522.
- 784 [9] Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B.
785 (2016). Learning to branch in mixed integer programming. In *AAAI*
786 *Conference on Artificial Intelligence*, pages 724–731.
- 787 [10] Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring poly-
788 nomials with rational coefficients. *Mathematische Annalen*, 261(4):515–
789 534.

- 790 [11] Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and
791 advancing SAT solving with graph neural networks. *Transactions on*
792 *Machine Learning Research*.
- 793 [12] Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algo-
794 rithm for propositional satisfiability. *IEEE Transactions on Computers*,
795 48(5):506–521.
- 796 [13] Millidge, B. (2022). Deconfusing direct vs amortized opti-
797 mization. Unrefereed technical note, [https://www.beren.io/](https://www.beren.io/2022-09-25-Deconfusing-direct-vs-amortized-optimization/)
798 [2022-09-25-Deconfusing-direct-vs-amortized-optimization/](https://www.beren.io/2022-09-25-Deconfusing-direct-vs-amortized-optimization/).
- 799 [14] Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., and Malik, S.
800 (2001). Chaff: Engineering an efficient SAT solver. In *Design Automa-*
801 *tion Conference*, pages 530–535.
- 802 [15] Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via
803 lazy clause generation. *Constraints*, 14(3):357–391.
- 804 [16] Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Im-
805 proved practical algorithms and solving subset sum problems. *Mathe-*
806 *matical Programming*, 66(1–3):181–199.
- 807 [17] Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill,
808 D. L. (2019). Learning a SAT solver from single-bit supervision. In *In-*
809 *ternational Conference on Learning Representations*.
- 810 [18] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Ben-
811 gio, Y. (2018). Graph attention networks. In *International Conference*
812 *on Learning Representations*.
- 813 [19] Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2019). How powerful
814 are graph neural networks? In *International Conference on Learning*
815 *Representations*.